



PulpoCon 2025

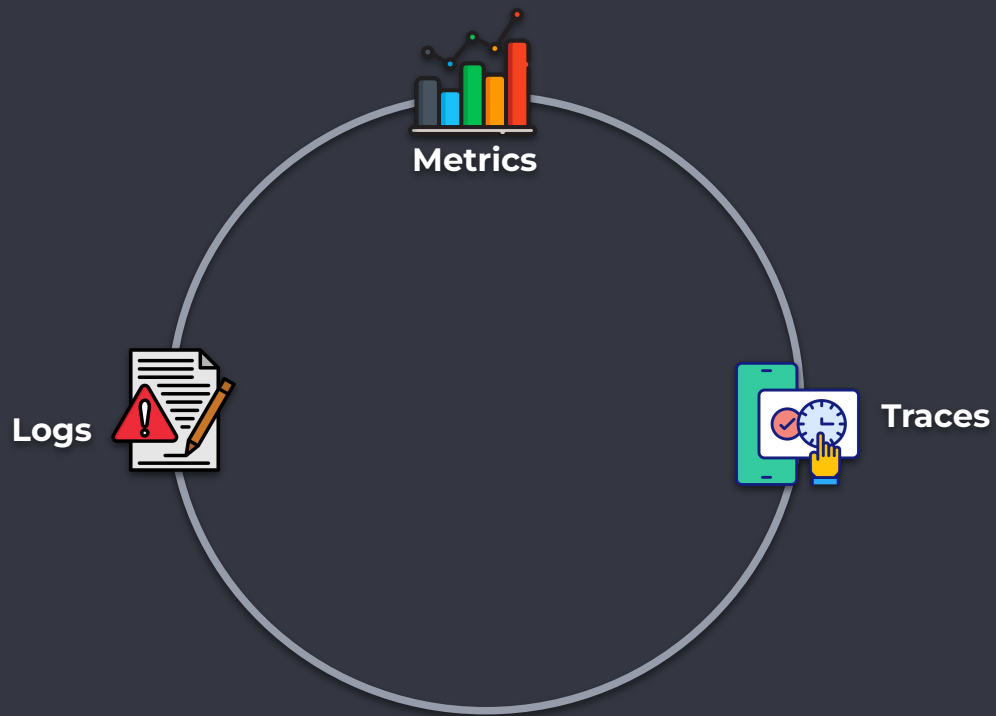
Unknown unknowns with Continuous Profiling

Roger Coll | Engineer @ Elastic

[Draft] Agenda

1. Observability pillars
2. Profiles specification
3. Profiling Agent architecture
4. Stack unwinding
5. Supported languages
6. Symbolization
7. Demo: Detecting unknown-unknowns of the OpenTelemetry Demo with Devfiler and the OpenTelemetry eBPF Profiler

The Three pillars of Observability



What's happening in our systems?

```
{ http.error.rate = 75%, timestamp = ... }
```

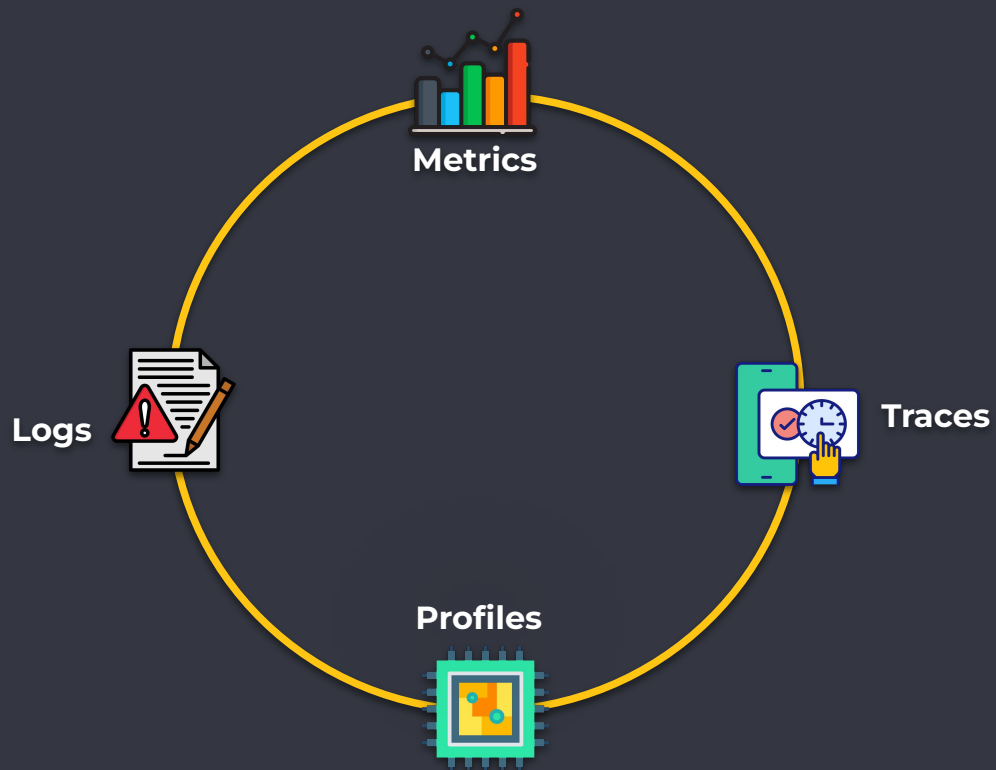
Why is it happening?

```
{ message = "user not found", timestamp = ... }
```

Where is it happening?

```
{ name = "get_user", span_id = ..., parent_id = ... }
```

The ~~Three~~ Four pillars of Observability



Understand CPU usage **quickly** and **completely**

Unknown unknowns

- Why is the kernel on-CPU so much? What code-paths?
- Are the CPUs stalled on memory I/O?
- Which code-paths are allocating memory, and how much?
- Is a certain kernel function being called, and how often?
- What reasons are threads leaving the CPU?
- Regressions?

OpenTelemetry Profiles

[OpenTelemetry announces support for profiling](#)

```
dictionary:
  attributeTable:
    - key: process.executable.build_id.htlhash
      value:
        stringValue: 600DCAFE4A110000F2BF38C493F5FB92
    - key: profile.frame.type
      value:
        stringValue: native
    - key: host.id
      value:
        stringValue: localhost
  locationTable:
    - address: "111"
      attributeIndices:
        - 1
      mappingIndex: 0
  mappingTable:
    - attributeIndices:
        - 0
  stringTable:
    - samples
    - count
    - cpu
    - nanoseconds
  resourceProfiles:
    - resource: {}
      scopeProfiles:
        - profiles:
            - attributeIndices:
                - 2
              periodType:
                typeStrindex: 2
                unitStrindex: 3
              sample:
                - locationsLength: 1
                  timestampsUnixNano:
                    - "0"
              sampleType:
                - unitStrindex: 1
            scope: {}
```

To Other Signals

```
profile:  
  link:  
    trace_id: 12345  
    span_id: 67890  
  ...
```

From Other Signals

```
log:  
  profile_id: 88888  
  trace_id: 12345,  
  span_id: 67890,  
  severity_text: "DEBUG",  
  body: "Testing log",  
  ...
```

OpenTelemetry Profiling Agent



Frictionless deployment, no code changes or app restarts, powered by eBPF.



Whole-system visibility: from the Kernel through user space.



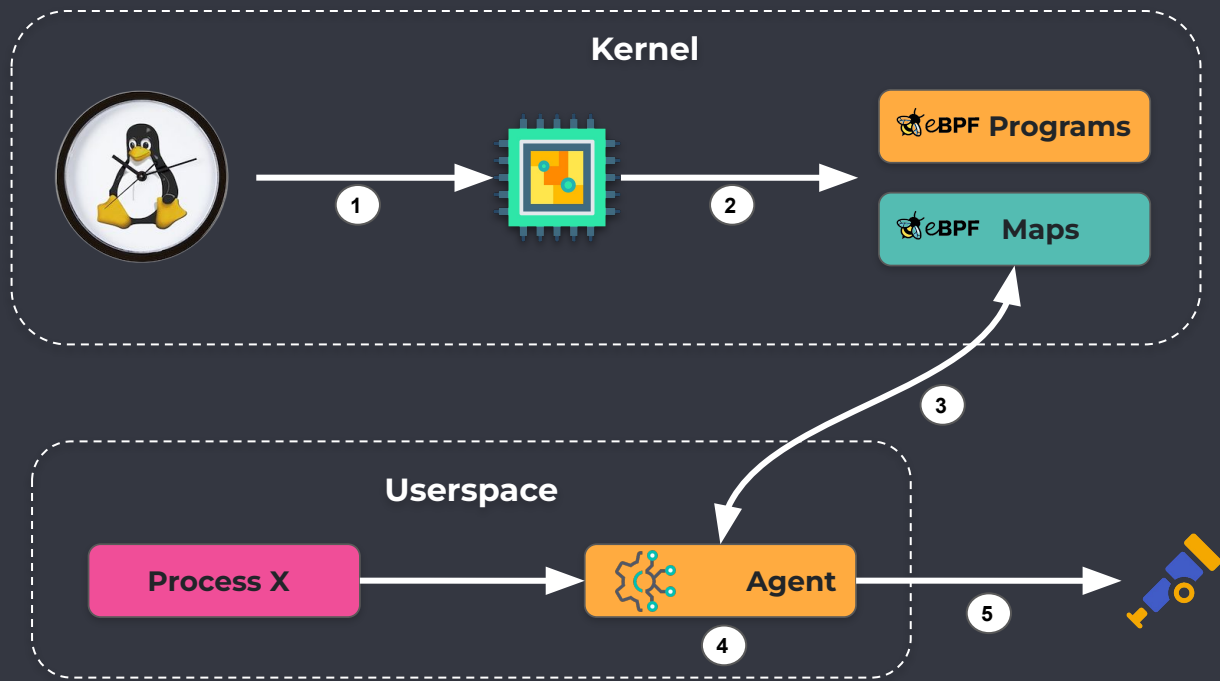
Multiple High Level Language runtimes.



Extremely Low Overhead: aiming for: < 1% system CPU, ~250MB of RAM

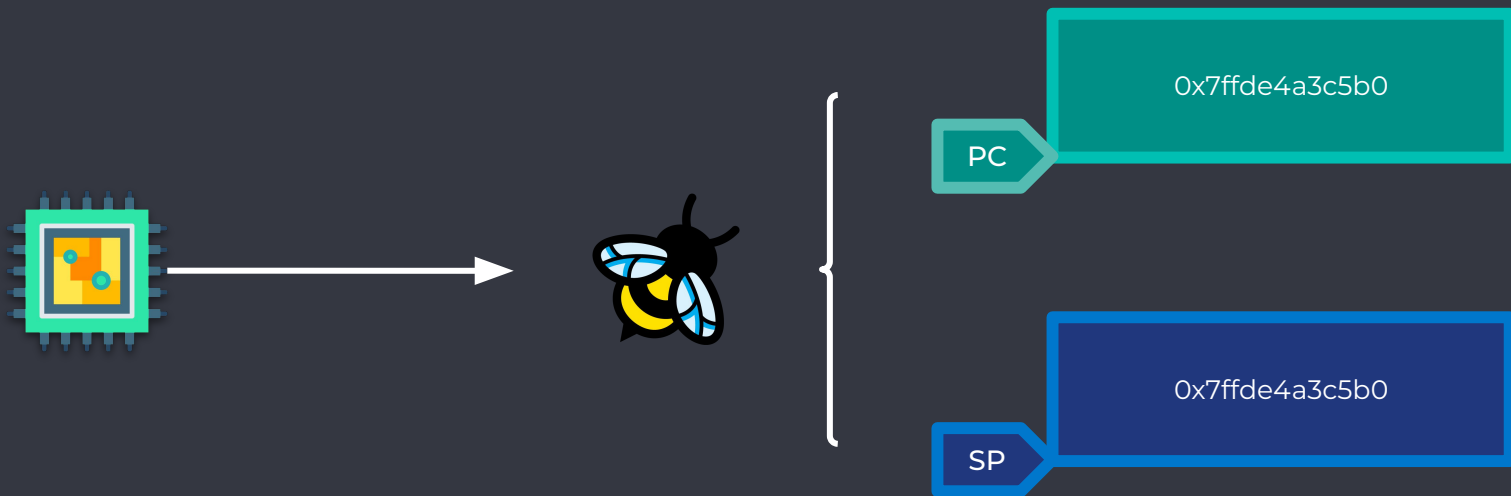
[Elastic donates profiling agent to OpenTelemetry](#)

Profiling Agent Architecture

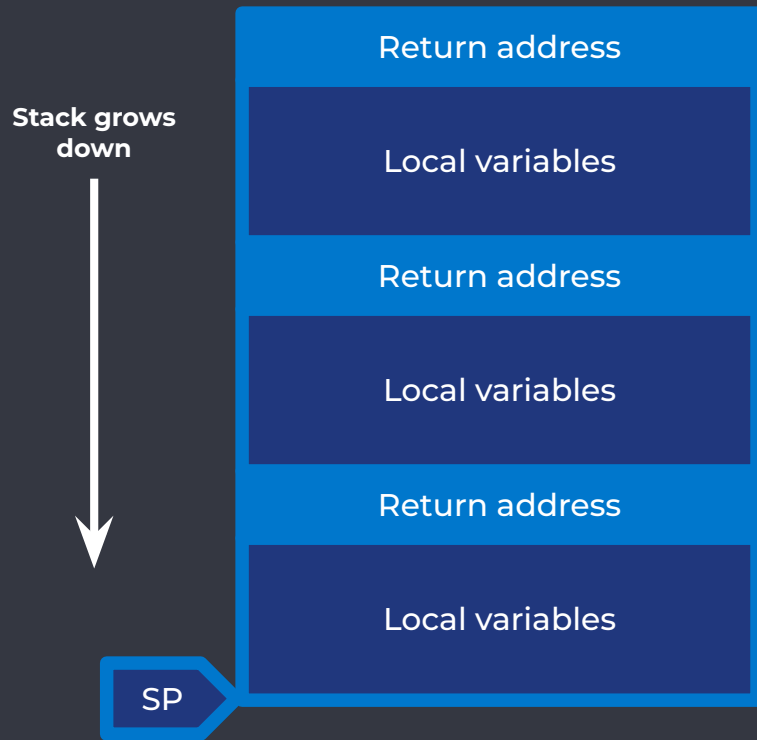


- 1 CPU interrupt (19Hz)
- 2 Stack unwinding
- 3 Process-specific data
- 4 Symbolization
- 5 OTLP Profiles

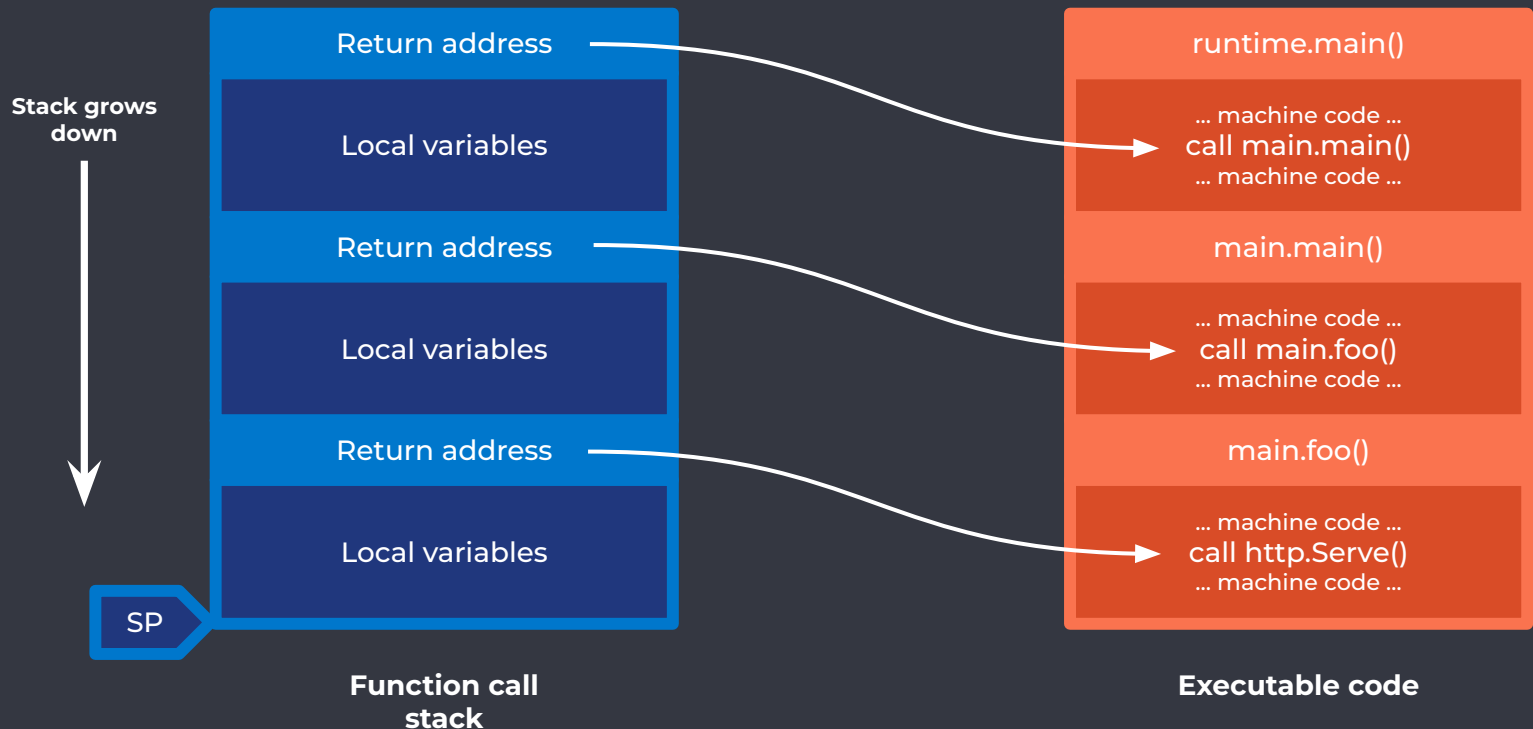
Stack unwinding (FPO)



Stack unwinding (FPO)



Stack unwinding (FPO)



Stack unwinding

Native (C, C++, Go, ...) vs High Level Languages (Java, Python, ...)

Native executables: via exception handling and unwinding mechanism (.eh_frame) on ELF's metadata for C++, Rust, C, etc. For Go, unwinding relies on the binary [.gopclntab](#) section.

```
String dump of section '.gopclntab':
```

```
...
```

```
[19140f] github.com/collector/receiver/xreceiver.NewFactory
```

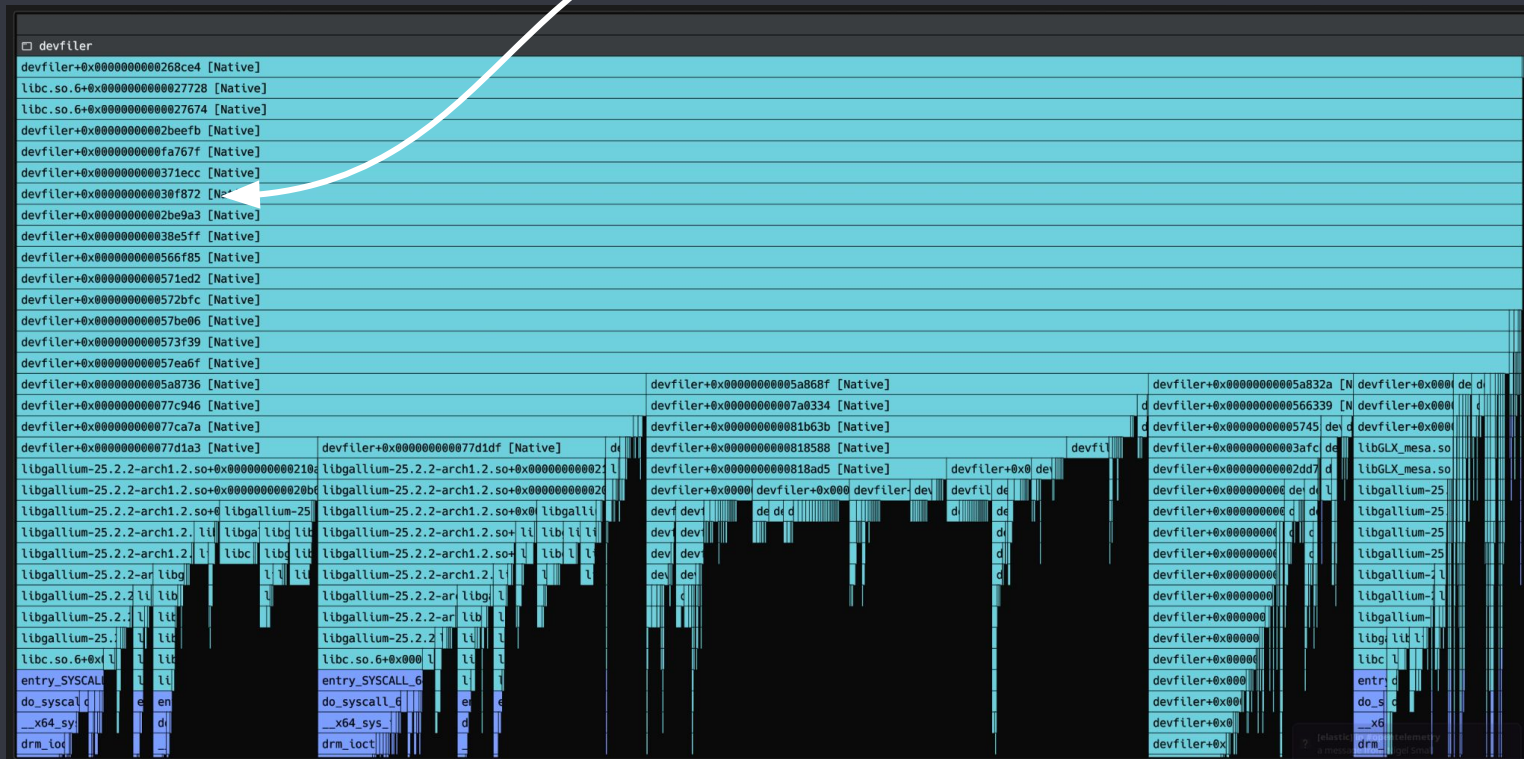
```
[19144b] github.com/collector/receiver.NewFactory
```

```
[19147d] github.com/collector/receiver/xreceiver.factory.CreateDefaultConfig
```

HLL runtimes: Every language needs to keep track of a call stack.

- Reverse engineer how each HLL runtime does it.
- Prepare eBPF maps with struct offsets and other information to enable unwinding.
- Supporting new versions can be a matter of only updating a few offsets

foo() or bar() ? 🤔



Symbolization

Stack trace offsets

```
moduleID1+0x6c8  
moduleID1+0x9c9  
moduleID2+0xc8714  
moduleID2+0x281e1
```



Human-readable symbols

```
vmlinux: __x64_sys_nanosleep  
vmlinux: do_syscall_64  
pf-host-agent: runtime.usleep  
pf-host-agent: runtime.runqgrab
```

Replace addresses (offsets) with symbol names (also source filenames, line numbers when available)

Symbolization

Native vs High Level Languages

Native executables: may be symbolized on-target (if symbols always present) or post-collection (e.g. at the backend)

HLL runtimes: always present in target process memory space. The agent can perform symbolization on-target.



unwinders

```
moduleID1+0x6c8  
moduleID1+0x9c9  
moduleID2+0xc8714  
moduleID2+0x281e1
```

Stack trace offsets



process
memory space

```
V8::EntryFrame [JS]  
V8::StubFrame [JS]  
getPlaywrightVersion in /usr/local/..  
V8::ExitFrame [JS]
```

Supported languages

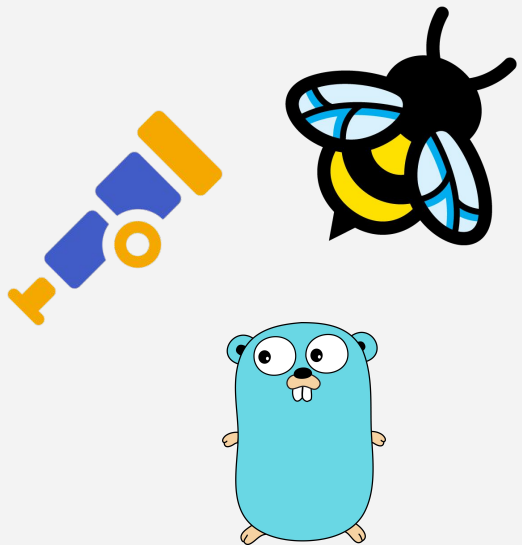
*DotNet is not supported on ARM64



[illegible]

- 

Demo



- DevFiler + Go (on-target) and Rust (post-collection) service's symbolization
- OpenTelemetry Astronomy Shop Demo deployments

¡Gracias!



[opentelemetry-ebpf-profiler](#)



[opentelemetry-proto](#)



[elastic/devfiler](#)